SOP Versioning and Unreal Engine Build Automation

Author: Aryo, CTO
Target Audience: DevOps, Lead Programmer, CTO

Objective: Panduan setup "Zero to Hero" untuk otomasi build Unreal Engine menggunakan GitHub Actions (Self-Hosted) dengan pemisahan jalur Production dan Staging, serta standarisasi Git Branching.

1. ARCHITECTURE OVERVIEW

Sistem ini menggunakan dua jalur otomasi terpisah untuk menghemat resource PC Build:

1. **Jalur Staging (Manual Trigger):** Digunakan untuk testing harian. CTO/Lead masuk ke tab GitHub Actions, memilih branch `staging` atau `feature-x`, lalu menekan tombol "Run". Hasil build disimpan sebagai *temporary artifact* (bertahan 90 hari).
2. **Jalur Production (Release Trigger):** Digunakan untuk rilis resmi ke Client/End-User. Terpicu otomatis saat CTO membuat "GitHub Release" dan menempelkan tag `v1.x.x`. Hasil build permanen terlampir di halaman rilis.

2. DETAILED SETUP (LAKUKAN SEKALI DI AWAL)

A. Persiapan PC Runner (Mesin Lokal Studio)

PC ini akan bekerja berat. Jangan gunakan PC yang sedang dipakai kerja aktif oleh artist/programmer.

1. Install **Unreal Engine** (Pastikan versinya sama persis dengan project).
2. Install **Visual Studio 2022** > Pilih workload **"Game development with C++"**.
3. Install **Git for Windows**.
4. Buka PowerShell/Git Bash, login ke akun GitHub yang memiliki akses ke repository agar PC ini bisa melakukan `git pull` tanpa prompt password.

B. Registrasi Self-Hosted Runner di GitHub

1. Buka halaman Repository project kamu di GitHub.com.
2. Buka tab **Settings** > Lihat menu kiri bawah, klik **Actions** > **Runners**.
3. Klik tombol hijau **New self-hosted runner**.
4. Pilih OS **Windows** dan Architecture **x64**.

5. Buka aplikasi **PowerShell** di PC Runner lokal kamu.
6. Copy-paste perintah yang muncul di GitHub satu per satu ke PowerShell.
   - *Tahap Configure:* Saat ditanya nama runner, ketik `Studio-Build-PC`.
7. **PENTING:** Saat ditanya *"run as service"*, ketik `Y` (Yes). Ini agar runner otomatis menyala di background setiap kali PC dihidupkan.

C. Konfigurasi Discord Webhook (Notifikasi)

1. Buka Discord > Masuk ke Server Studio > Buka pengaturan *Channel* khusus notif build.
2. Masuk ke **Integrations** > **Webhooks** > Klik **New Webhook**.
3. Beri nama "UE Build Bot", lalu klik **Copy Webhook URL**.
4. Kembali ke GitHub Repository > **Settings** > **Secrets and variables** > **Actions**.
5. Klik **New repository secret**.
   - Name: `DISCORD_WEBHOOK`
   - Secret: *(Paste URL Discord tadi ke sini)*

3. STANDAR BRANCHING GIT

Untuk memastikan CI/CD berjalan sesuai jalurnya dan mencegah bentrok kode, seluruh tim WAJIB mengikuti standar cabang (branch) berikut:

- `main` / `master` **(Production Branch)**
  - **Fungsi:** Cabang paling krusial. Berisi kode yang 100% stabil dan siap rilis ke publik.
  - **Aturan:** *Protected Branch*. Dilarang melakukan *commit* langsung. Hanya menerima *Pull Request* (PR) yang sudah di-review dari cabang `staging` atau `hotfix`.
  - **Trigger CI/CD:** Membuat *Release* dari cabang ini akan memicu **Production Build Otomatis**.
- `staging` / `develop` **(Integration Branch)**
  - **Fungsi:** Pusat integrasi kode harian. Tempat berkumpulnya fitur dari berbagai programmer.
  - **Aturan:** Cabang utama (*default*) untuk QA Tester melakukan uji coba.
  - **Trigger CI/CD:** Target utama untuk di-build melalui tombol **Manual Staging Build**. PENTING: Pastikan branch ini sudah di-push ke GitHub sebelum menjalankan

Manual Build.

- `feature/[nama-fitur]` **(Working Branch)**
  - **Fungsi:** Tempat programmer bekerja membuat fitur sehari-hari (contoh: `feature/inventory-system`).
  - **Aturan:** Harus selalu dibuat bercabang (*branch out*) dari `staging`. Setelah fitur selesai, programmer wajib membuat *Pull Request* untuk menggabungkannya kembali ke `staging`.
- `hotfix/[nama-bug]` **(Emergency Branch)**
  - **Fungsi:** Cabang darurat untuk memperbaiki *bug* kritis di versi rilis / production secepatnya.
  - **Aturan:** Dibuat bercabang langsung dari `main`. Setelah fix selesai, **WAJIB** di-merge kembali ke `main` **DAN** ke `staging` agar *bug* tidak terulang di rilis berikutnya.

4. IMPLEMENTASI SCRIPT YAML (PIPELINE)

Kita akan membuat dua file `.yml` di dalam folder project lokal kamu, tepatnya di dalam folder `.github/workflows/`. Setelah dibuat, lakukan *Commit* dan *Push* ke GitHub.

SCRIPT 1: Staging/Dev Build (Tombol Manual)

Buat file: `.github/workflows/build_staging.yml`

```yaml
name: MANUAL BUILD - Staging/Dev

on:
  workflow_dispatch:
    inputs:
      branch_target:
        description: 'Pilih Branch yang mau di-build (contoh: staging, develop)'
        required: true
        default: 'staging'

env:
  FORCE_JAVASCRIPT_ACTIONS_TO_NODE24: true

jobs:
  build_staging:
    runs-on: self-hosted
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4
        with:
          ref: ${{ github.event.inputs.branch_target }}

      - name: Setup Variables
        run: |
          $DATE_STR = Get-Date -Format "yyyyMMdd-HHmm"
          $REPO_NAME = "${{ github.event.repository.name }}"
          echo "ZIP_NAME=${REPO_NAME}_Staging_${DATE_STR}.zip" >> $env:GITHUB_ENV
          echo "BUILD_DIR=${{ github.workspace }}\Build_Output" >> $env:GITHUB_ENV

      - name: Run Unreal Automation Tool
        run: |
          $UAT_Path = "C:\Program Files\Epic
Games\UE_5.3\Engine\Build\BatchFiles\RunUAT.bat"
          $ProjectFile = Get-ChildItem -Path "${{ github.workspace }}" -Filter
*.uproject | Select-Object -First 1
          if (-not $ProjectFile) { throw "No .uproject file found!" }
          & $UAT_Path BuildCookRun -project="$($ProjectFile.FullName)" -noP4 -
platform=Win64 -clientconfig=Development -cook -allmaps -build -stage -pak -archive -
archivedirectory="${{ env.BUILD_DIR }}"

      - name: Compress Build Output
        run: Compress-Archive -Path "${{ env.BUILD_DIR }}\Windows\*" -DestinationPath
"${{ env.ZIP_NAME }}" -Force

      - name: Upload Artifact to GitHub
        uses: actions/upload-artifact@v4
        with:
          name: ${{ env.ZIP_NAME }}
```

```
          path: ${{ env.ZIP_NAME }}
          retention-days: 7

      # Menggunakan PowerShell Native untuk mem-bypass limitasi Docker di Windows
Runner
      - name: Notify Discord
        if: always()
        run: |
          $payload = @{
            content = "**🛠️ STAGING BUILD Selesai!**`n**Project:** ${{
github.event.repository.name }}`n**Branch:** `${{ github.event.inputs.branch_target
}}` `n**Status:** ${{ job.status }}`n*Silakan cek tab Actions di GitHub untuk
mendownload Artifact-nya.*"
          }
          Invoke-RestMethod -Uri "${{ secrets.DISCORD_WEBHOOK }}" -Method Post -Body
($payload | ConvertTo-Json) -ContentType "application/json"
```

SCRIPT 2: Production Build (Otomatis via Release)

Buat file: `.github/workflows/build_production.yml`

```yaml
name: AUTO BUILD - Official Release

on:
  release:
    types: [published]

env:
  FORCE_JAVASCRIPT_ACTIONS_TO_NODE24: true

jobs:
  build_production:
    runs-on: self-hosted
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Setup Variables
        run: |
          $TAG_NAME = "${{ github.event.release.tag_name }}"
          $REPO_NAME = "${{ github.event.repository.name }}"
          echo "ZIP_NAME=${REPO_NAME}_${TAG_NAME}.zip" >> $env:GITHUB_ENV
          echo "BUILD_DIR=${{ github.workspace }}\Build_Output" >> $env:GITHUB_ENV

      - name: Run Unreal Automation Tool
        run: |
          $UAT_Path = "C:\Program Files\Epic
Games\UE_5.3\Engine\Build\BatchFiles\RunUAT.bat"
          $ProjectFile = Get-ChildItem -Path "${{ github.workspace }}" -Filter
*.uproject | Select-Object -First 1
          if (-not $ProjectFile) { throw "No .uproject file found!" }
          & $UAT_Path BuildCookRun -project="$($ProjectFile.FullName)" -noP4 -
platform=Win64 -clientconfig=Shipping -cook -allmaps -build -stage -pak -archive -
archivedirectory="${{ env.BUILD_DIR }}"

      - name: Compress Build Output
        run: Compress-Archive -Path "${{ env.BUILD_DIR }}\Windows\*" -DestinationPath
"${{ env.ZIP_NAME }}" -Force

      - name: Upload Build to Release
        uses: svenstaro/upload-release-action@v2
        with:
          repo_token: ${{ secrets.GITHUB_TOKEN }}
          file: ${{ env.ZIP_NAME }}
          asset_name: ${{ env.ZIP_NAME }}
          tag: ${{ github.ref }}
          overwrite: true

      # Menggunakan PowerShell Native untuk mem-bypass limitasi Docker di Windows
```

```
Runner
      - name: Notify Discord
        if: always()
        run: |
          $payload = @{
            content = "**🚀 PRODUCTION RELEASE Selesai!**`n**Project:** ${{
github.event.repository.name }}`n**Version:** `${{ github.event.release.tag_name }}`
`n**Status:** ${{ job.status }}`n**Download Link:** ${{ github.event.release.html_url
}}"
          }
          Invoke-RestMethod -Uri "${{ secrets.DISCORD_WEBHOOK }}" -Method Post -Body
($payload | ConvertTo-Json) -ContentType "application/json"
```

5. CARA PENGGUNAAN (EKSEKUSI)

Cara Build Staging (Manual)

1. **PENTING:** Pastikan branch `staging` sudah di-push (berada) di server GitHub.
2. Buka repository GitHub, klik tab **Actions**.
3. Di sebelah kiri, di bawah tulisan *Workflows*, klik **MANUAL BUILD - Staging/Dev**.
4. Di sebelah kanan, klik tombol **Run workflow**.
5. Ketik nama branch yang mau di build (misal: `staging` atau `feature-multiplayer`).
6. Klik tombol hijau **Run workflow**.
7. *Cara Download:* Setelah selesai, klik riwayat run tersebut, gulir ke paling bawah ke bagian **Artifacts**. Klik file zip untuk mendownloadnya.

Cara Build Production (Otomatis)

1. Buka repository GitHub, pastikan branch `main` sudah mendapatkan Pull Request terbaru dari `staging`.
2. Lihat sidebar sebelah kanan, klik **Releases** > **Draft a new release**.
3. Buat Tag baru (misal: `v1.0.0-beta`). Pastikan Target Branch mengarah ke `main`.
4. Isi judul dan klik **Publish release**.
5. Script "AUTO BUILD - Official Release" akan otomatis berjalan dan menempelkan file `.zip` ke halaman release tersebut jika sudah selesai.